4e. Query FSR Tool Specs

Query FSR — Tool Spec

REST Tool: `query_fsr`
Spec Owner: DS team **Impl Owner:** DEV team
Date: 2026-03-05

Summary

This document describes the target production behavior for a future `query_fsr` REST tool. The cleaned `fsr_pipeline_dbr_v6` package in this repo is still an experimental Databricks notebook workflow used to validate ingestion and retrieval quality; it does not yet implement this full service contract.

The Query FSR tool performs hybrid retrieval (semantic + keyword) over pre-chunked FSR (Field Service Report) documents whose embeddings are generated with a self-managed Azure OpenAI embedding model. Those embeddings are persisted in the Delta source table and indexed in Databricks Vector Search. At query time the tool accepts only the user query text, generates the matching `query_vector` internally with the same embedding model, and supplies both `query_text` and `query_vector` to Vector Search. It returns the top-k most relevant chunk records after reranking, enriched with structured metadata from three additional Databricks SQL views.

The standalone `query_fsr_with_metadata.py` implementation validated an important operational detail that this spec now makes explicit: the initial Vector Search call should return only the minimal chunk-identifying columns needed for ranking and traceability, and metadata enrichment should happen afterward only for the returned chunk IDs. The grouped response shape is therefore a second-stage assembly step, not a raw Vector Search response.

The tool is exposed as a **REST endpoint only** — no MCP endpoint is required. This reflects the current architecture decision to implement all tool interfaces as REST for the time being.

Implementation References

- **Entire code package:** `Here`
- **Ingestion code package:** fsr_pipeline_dbr_final
- **Grouped response reference implementation:** `query_fsr_with_metadata.py`

Retrieval Validation Snapshot

Source: `docs/fsr_retrieval_metrics_final.csv`.

- 16 retrieval variants were compared across custom and Databricks-built-in approaches, with and without criteria text appended to the query.
- Best low-k Databricks-native result: **DBR built-in HYBRID + criteria** achieved **Recall@1 = 45.5**, **Recall@5 = 67.4**, and **Recall@10 = 78.8**.
- Best higher-k Databricks-native result: **DBR built-in Reranked + criteria** achieved **Recall@20 = 86.4**.
- Best overall experimental Recall@10: **Custom Hybrid_70_30 without criteria** reached **81.8**.
- Best overall experimental Recall@20: **Custom Hybrid_70_30 with criteria** reached **88.6**.
- Production implication: **Databricks native HYBRID + criteria** is the strongest low-latency baseline from this experiment, while reranking remains worth evaluating when deeper recall matters and additional latency is acceptable.

Data Sources

Primary: Vector Search Table (Chunks + Embeddings)

- **Production target table:** `vaid.ai_std_con_field_service_report.field_service_report`
- **Production target VS Index:** `vaid.ai_std_con_field_service_report.vs_field_service_report` (self-managed embedding index)
- **Current experiment full-corpus table:** `main.gp_services_sdg_poc.field_service_report`
- **Current experiment 30-ESN LiteLLM subset table/index:** `main.gp_services_sdg_poc.field_service_report_gt_litellm` / `main.gp_services_sdg_poc.vs_field_service_report_gt_litellm`
- **Why the LiteLLM subset exists:** it was created as a focused 30-ESN ground-truth evaluation set for REChain retrieval testing and self-managed LiteLLM embedding validation. It is not the full FSR corpus.
- **Use these broader objects when you need more coverage:** `main.gp_services_sdg_poc.field_service_report` contains the larger chunk corpus, and `vaid.ai_std_con_field_service_report.vs_field_service_report` is the broader `vs_field_service_report` index described in this spec.
- **VS Endpoint:** `pw-ser-sdg-vector-search`
- **Embedding model:** self-managed Azure OpenAI embedding deployment via LiteLLM (current runtime default: `azure-text-embedding-3-large-1`, 3072-dim)
- **Embedding persistence:** document embeddings are generated during ingestion, stored in the Delta table, and indexed in Vector Search; query embeddings are generated by the tool during retrieval

Column	Type	Notes

chunk_id	STRING	PK — format: {pdf_name}_{chunk_index} or {pdf_name}_{chunk_index}__{esn} for multi-ESN chunks
pdf_name	STRING	Source PDF identifier (GUID) — **join key** to fsr_pdf_ref.s3_filename
page_number	INT	Chunk start page within the document, as stored by the current pipeline (not chunk ordinal)
generator_serial	STRING	ESN identified from report text via LLM + regex extraction (nullable). When a chunk is associated with multiple ESNs, the row is **duplicated** — one row per ESN — so that a VS equality filter on generator_serial surfaces the chunk for every tagged unit.
report_date	DATE	Report date extracted during ingestion (nullable)
chunk_text	STRING	The chunked text content (up to ~4 000 tokens)
embedding	ARRAY	Self-managed Azure OpenAI embedding written during ingestion and indexed in Vector Search. This field is not returned to the caller.
created_at	TIMESTAMP	Row creation time
uploaded_at	TIMESTAMP	Row upload time

Supplementary: Structured Metadata Views (3 tables)

All three views are joined to the vector table results to enrich each returned chunk with structured FSR metadata.

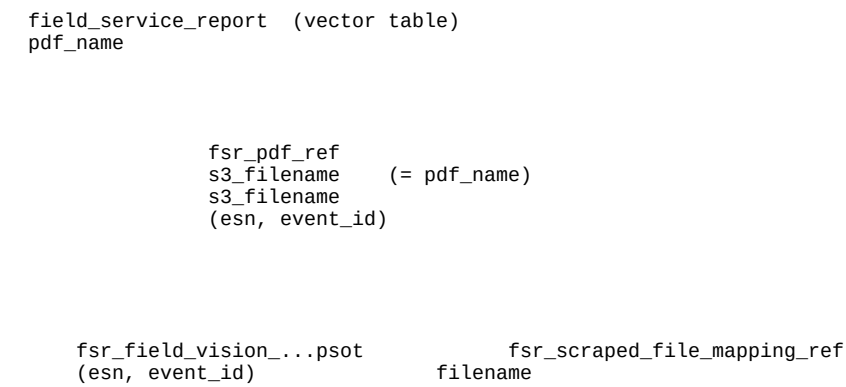
#	Alias	Full Table Name	Description
1	**FSR Report**	vgpd.fsr_std_views.fsr_field_vision_field_services_report_psot	Comprehensive field-services and business-unit details for outages — captures outage type, scope of work, site, emergency classification, equipment fired hours, start/end dates, and operational metrics. ~35,251 rows.
2	**PDF Ref**	vgpd.fsr_std_views.fsr_pdf_ref	Mapping of FSR documents in FieldVision to PDF files using FieldVision application metadata. Links report IDs to physical PDF filenames.
3	**Scraped Mapping**	vgpd.fsr_std_views.fsr_scraped_file_mapping_ref	Metadata extracted from Field Services Report PDFs via text scraping. Provides supplementary file-level metadata not available in FieldVision.

First-pass Vector Search return columns

The metadata-enrichment implementation only needs a small set of columns from the initial Vector Search query before it hydrates the full chunk row from the Delta table.

Column	Purpose
chunk_id	Primary identifier used to reload the full chunk row from the chunk table
pdf_name	Filename join key for PDF metadata lookups
page_number	Page anchor when the full chunk row is unavailable
chunk_text	Retrieval text shown to callers and optionally reranked
generator_serial	ESN filter key and tie-break input for PDF metadata selection

Join Relationships



Join	From	To	Key
	`field_service_report.pdf_name`	`fsr_pdf_ref.s3_filename`	PDF filename (GUID)
	`fsr_pdf_ref.s3_filename`	`fsr_scraped_file_mapping_ref.filename`	PDF filename
	`fsr_pdf_ref.(esn, event_id)`	`fsr_field_vision_...psot.(esn, event_id)`	Equipment serial + event

> ***Note on Join :*** *The `(event_id, esn)` composite key is not perfectly unique in the `_psot` table. Production implementation should explicitly deduplicate by preferring the row with the most advanced `report_unit_status` (Completed > Started > Not Started > Hold).*

> ***Filename normalization note:*** *the implementation had to normalize filename/path variants when joining (`guid`, `guid.pdf`, and path-qualified names). Production code should join on a normalized PDF stem, not on raw string equality alone.*

> ***PDF Ref selection note:*** *when multiple `fsr_pdf_ref` rows exist for the same PDF, prefer an exact `(pdf_name, esn)` match over filename-only matches before selecting the row used for downstream `_psot` enrichment.*

Inputs

Parameter	Type	Required	Default	Description
`equipment_serial_number`	string	**Yes**	—	ESN to scope retrieval. Only chunks belonging to FSRs for this ESN are considered.
`query`	string	**Yes**	—	Natural-language search query (e.g., "`vibration issues during startup`"). Drives both semantic and keyword legs of hybrid retrieval.
k	integer	**Yes**	—	Number of top results to return. This is the final output size.
`query_type`	string	No	`HYBRID`	Retrieval mode. Current implementation supports `HYBRID` and `ANN`. Production may choose to keep this fixed to `HYBRID`.

> *The public API does not accept `query_vector`. Callers provide `query` text only; the service computes the vector internally with the configured Azure OpenAI embedding model.*

Input Validation

Condition	Error
`equipment_serial_number` is empty or missing	`400 Bad Request` — "equipment_serial_number is required"
`query` is empty or missing	`400 Bad Request` — "query is required"
k is missing or `0`	`400 Bad Request` — "k must be a positive integer"
`query_type` is provided but unsupported	`400 Bad Request` — "query_type must be one of: HYBRID, ANN"

Retrieval Pipeline

Step 1 — ESN-scoped candidate set

Filter the vector table to only include records matching the provided `equipment_serial_number` (via equality filter on `generator_serial`).

Step 2 — Generate query vector internally

Embed the provided `query` string with the configured self-managed Azure OpenAI embedding model. The caller does not provide `query_vector`; the service derives it at request time using the same embedding family used during ingestion.

Step 3 — Vector Search retrieval (top k)

Execute a native Databricks Vector Search query against the self-managed VS index, passing both `query_text` and the internally generated `query_vector`.

For `HYBRID`, Databricks search combines:

- **Dense retrieval** — vector similarity against the supplied `query_vector`, generated from the Azure OpenAI embedding model
- **Learned keyword search** — BM25-style keyword matching against `chunk_text` using the original `query_text`

Results are fused internally via **Reciprocal Rank Fusion (RRF)** and returned with scores normalised 0–1.

The initial Vector Search request should return the minimal retrieval columns needed for downstream assembly:

- `chunk_id`
- `pdf_name`
- `page_number`
- `chunk_text`
- `generator_serial`

Step 4 — Reranking (top k)

Apply the **Databricks built-in reranker** (`DatabricksReranker`, reranking on `chunk_text`) to the hybrid search results. Return the top `k` results.

Step 5 — Chunk row hydration

Reload the full chunk-table rows for the returned `chunk_id` values before metadata enrichment. This allows the response to expose additional chunk columns such as `report_date`, `created_at`, and `uploaded_at` without overloading the initial Vector Search return set.

If a full chunk-table row cannot be found for a returned `chunk_id`, the service should still return a fallback `chunk` object assembled from the first-pass Vector Search row.

Step 6 — Metadata enrichment

For each of the returned chunks, JOIN only those chunk rows to the three supplementary views (via the join relationships above) and include all columns from all four tables in the response.

The implementation showed that these follow-up lookups can be parallelized safely because they depend only on the returned `chunk_id` and `pdf_name` sets.

Sample Query Code (Service Internal)

External callers to `query_fsr` submit only `equipment_serial_number`, `query`, and `k`. The service generates `query_vector` internally before calling Vector Search.

```
import json
import requests

from your_service.vector_embeddings import get_query_vector

# Query details
equipment_serial_number = "270T658"
query_text = "What is the status of the equipment?"
k = 3
query_type = "HYBRID"

query_vector = get_query_vector(query_text)

response = requests.post(
  "https://<workspace-host>/api/2.0/vector-search/indexes/vaid.ai_std_con_field_service_report.
vs_field_service_report/query",
  headers={
    "Authorization": "Bearer <databricks-token>",
    "Content-Type": "application/json",
  },
  json={
    "query_text": query_text,
    "query_vector": query_vector,
    "filters_json": json.dumps({"generator_serial": equipment_serial_number}),
    "num_results": k,
    "query_type": query_type,
    "columns": ["chunk_id", "pdf_name", "page_number", "chunk_text", "generator_serial"],
  },
  timeout=60,
)
response.raise_for_status()
results = response.json()
```

Expected Output

Response Structure

```
{
  "results": [
    {
      "rank": 1,
      "rerank_score": 0.847,
      "chunk": {
        "chunk_id": "02a29a81-c142-42f0-a29a-81c14282f00b_3",
        "pdf_name": "02a29a81-c142-42f0-a29a-81c14282f00b",
        "page_number": 3,
        "generator_serial": "297104",
        "report_date": "2023-04-15",
        "chunk_text": "During startup, vibration levels on bearing #2 exceeded ...",
        "created_at": "2026-02-20T14:30:00Z",
        "uploaded_at": "2026-02-20T14:30:00Z"
      },
      "fsr_report": {
        "id": "00EE9B30-3A51-4966-AD3D-E934BBC67960",
        "event_id": "202446",
        "esn": "297104",
        "report_name": "297104 - HMI Controls Callout",
        "site": "MUSTANG GENERATING STATION",
        "site_name": "MUSTANG GENERATING STATION",
        "customer_name": "GOLDEN SPREAD ELECTRIC COOPERATIVE, INC",
        "technology_type": "Gas Turbine",
        "outage_type": "CALL OUT",
        "start_date": "2019-09-17T04:00:00Z",
        "end_date": "2020-06-20T04:00:00Z",
        "executive_summary": null,
        "total_fired_hours": null,
        "total_starts": null,
        "...": "all remaining columns from _psot table"
      },
      "pdf_ref": {
        "rd_report_id": "00EE9B30-3A51-4966-AD3D-E934BBC67960",
        "filename": "02a29a81-c142-42f0-a29a-81c14282f00b",
        "esn": "297104",
        "event_start_date": "2019-09-17",
        "event_end_date": "2020-06-20",
        "...": "all columns from fsr_pdf_ref"
      },
      "scraped_mapping": {
        "filename": "02a29a81-c142-42f0-a29a-81c14282f00b",
        "...": "all columns from fsr_scraped_file_mapping_ref"
      }
    }
  ],
  "metadata": {
    "equipment_serial_number": "297104",
    "query": "vibration issues during startup",
    "k": 5,
    "query_type": "HYBRID",
    "total_chunks_for_esn": 142,
    "result_count": 5,
    "duration_ms": 1230
  }
}
```


Output Rules

- All columns from all four tables are included in each result record, grouped by source table.
- The grouped source objects are stable: each result contains `chunk`, `fsr_report`, `pdf_ref`, and `scraped_mapping` keys even when a join does not resolve.
- When metadata is missing for a returned chunk, the unmatched grouped object should be empty (`{}`) rather than omitted.
- Embedding-bearing columns such as `embedding` and `chunk_embedding` are excluded from the response payload even when they exist in the chunk table.
- Blank / null cells `null`.
- Literal `"n/a"` keep as string.

Current Experiment vs Production Spec

1. **Service shape:** the current repository contains Databricks notebooks (`run_pipeline.py`, `run_evaluation.py`) and helper modules, not a deployed `query_fsr` REST service.
2. **Retrieval path:** this spec assumes one production path (`generator_serial` filter internal Azure OpenAI query embedding HYBRID retrieval with supplied `query_vector` reranking enrichment). The current experiment evaluates four modes instead: ANN/HYBRID with reranking on/off.
3. **Enrichment scope:** this spec expects runtime joins to three supplementary views and grouped response objects. The current experiment only enriches the chunk table with `generator_serial` and `report_date` from `vgpd.fsr_std_views.fsr_pdf_ref`; it does not construct joined `fsr_report`, `pdf_ref`, and `scraped_mapping` response payloads.

4. **Metadata assembly:** the standalone `query_fsr_with_metadata.py` sample now demonstrates the grouped response assembly path end to end: first-pass Vector Search retrieval, full chunk-row hydration by `chunk_id`, filename-normalized joins to the three metadata views, and grouped JSON output.
5. **Object names and embedding config:** the production catalog/schema names in this spec are target-state values. The current runtime uses the broader chunk corpus in `main.gp_services_sdg_poc.field_service_report`, while the self-managed LiteLLM index currently used in REChain experimentation is `main.gp_services_sdg_poc.vs_field_service_report_gt_litellm`, which was created only for the 30-ESN ground-truth subset in `main.gp_services_sdg_poc.field_service_report_gt_litellm`. For broader retrieval coverage, refer to v `aid.ai_std_con_field_service_report.vs_field_service_report`. Query embeddings are produced via the configured LiteLLM /Azure OpenAI deployment.
6. **`page_number` semantics:** in the current pipeline `page_number` stores the chunk start page, and evaluation treats a hit as a cited page falling within `[page_number, page_number + PAGE_WINDOW]`. It is not a chunk ordinal.
7. **No-result behavior:** the standalone sample currently returns `200` with `results: []` plus diagnostic metadata such as `query_type`, `vs_index` `chunk_table`, and `total_chunks_for_esn`. Production may still choose the stricter `404` contract documented below.
8. **Service operations:** request validation, HTTP status codes, authenticated caller logging, latency metrics, and publication to Confluence are defined here as production requirements, but they are not implemented in the cleaned notebook package.

Endpoint

Endpoint Type	Purpose	Consumer
REST API	Service-to-service and agent-invoked FSR retrieval	Assistants (Risk Evaluation, Narrative, Event History), Q&A Agent

> ***No MCP endpoint*** *is required for this tool. The architecture decision is to implement all tool interfaces as REST for the time being.*

DEV team determines the specific URL path, HTTP method (recommended: `POST`), and authentication mechanism.

Query Logging

> *Every `query_fsr` call must log:*

Field	Description
Timestamp	ISO 8601 request time
User	Authenticated identity (JWT/Ping ID). If service-to-service, log calling service name.
ESN	`equipment_serial_number` provided in the request
Query	The search query text
k	Requested result count
Result count	Actual number of results returned (may be < k if fewer chunks exist for the ESN)
Duration (ms)	End-to-end latency
Errors	Error message on failure, `null` on success

Error Responses

Status	Condition	Body
`400`	Validation failure (see §Input Validation)	`{ "error": "<message>" }`
`404`	No chunks found for the given ESN	`{ "error": "No FSR chunks found for ESN {esn}", "result_count": 0 }`
`500`	Internal retrieval or join failure	`{ "error": "Internal server error", "detail": "<message>" }`
`503`	Vector Search endpoint unavailable	`{ "error": "Vector Search service temporarily unavailable" }`

Recent Implementation Learnings

- `query_fsr_with_metadata.py` confirmed that the extra metadata fields are not returned by Vector Search directly; they are assembled only after a second-stage SQL lookup.

- The most robust implementation pattern is: minimal Vector Search return columns -> chunk hydration by `chunk_id` -> metadata joins by normalized PDF key and `(esn, event_id)`.
- Joins against `fsr_pdf_ref` need filename normalization and an ESN-aware selection rule to avoid attaching the wrong report metadata when multiple rows share a PDF stem.
- `_psot` deduplication needs a deterministic tie-break beyond status priority; the implementation now also falls back to latest end/start date and then report name.
- The grouped response should omit embedding arrays and keep empty metadata groups as `{}` for schema stability.

Acceptance Criteria

- [ ] Tool spec includes inputs and expected output (see §Inputs and §Expected Output)
- [ ] Tool is callable via **REST endpoint** and returns enriched chunk results
- [ ] `equipment_serial_number` is mandatory; requests without it return `400`
- [ ] `k` is mandatory; requests without it return `400`
- [ ] `query` is mandatory; requests without it return `400`
- [ ] If `query_type` is exposed, only supported values (`HYBRID`, `ANN`) are accepted
- [ ] Hybrid retrieval uses Databricks native hybrid search (`query_type="HYBRID"`)
- [ ] Reranking uses Databricks built-in reranker on `chunk_text`
- [ ] Multi-ESN chunks are duplicated at ingestion time (one row per ESN) so `generator_serial` filter works for all tagged ESNs
- [ ] ESN extraction uses LLM + regex identification from report text
- [ ] Output includes all fields from all 4 tables: `field_service_report`, `fsr_field_vision_field_services_report_psot`, `fsr_pdf_re`, `fsr_scraped_file_mapping_ref`
- [ ] Joins correctly use `filename  pdf_name` and `(esn, event_id)` as documented
- [ ] Filename joins normalize path / extension variants before matching
- [ ] Returned `chunk` objects exclude embedding-bearing columns such as `embedding` and `chunk_embedding`
- [ ] Duplicate `(event_id, esn)` pairs in `_psot` are deduplicated (prefer Completed status)
- [ ] Query logs capture: **Timestamp**, **User**, **ESN**, **Query**, **k**, **Result count**, **Duration**, **Errors**
- [ ] Response `metadata` includes `k`, result count, and timing
- [ ] Documentation published to **Confluence** with full details